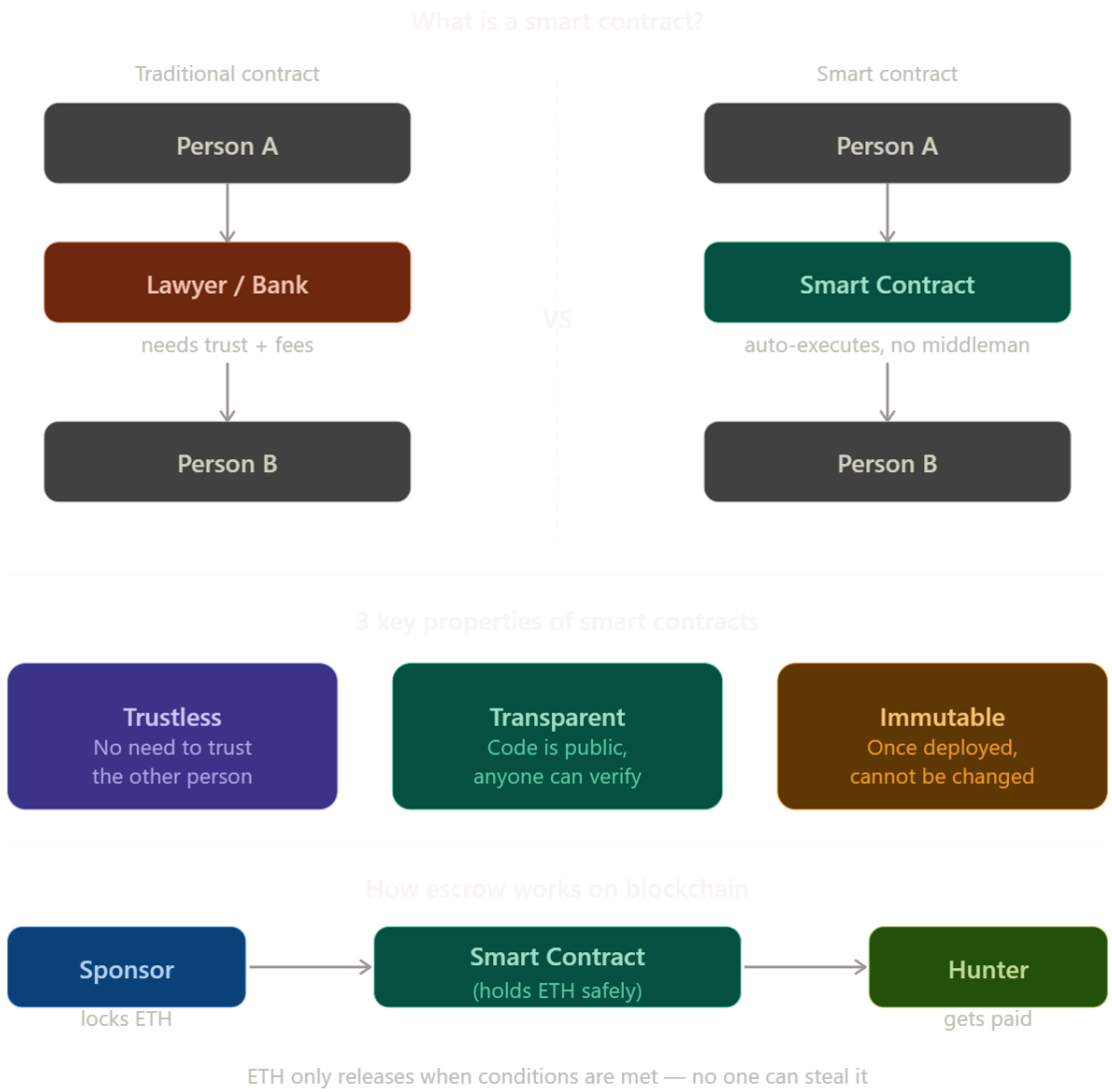
Pranav Titambe , D20A, 60

Blockchain Lab
EXP 6

Aim - Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory - Smart contracts are self-executing programs that live on a blockchain. Think of them like a vending machine — you put in money, the machine checks the rules, and automatically gives you what you paid for. No human in the middle.



Decentralized Bug Bounty Escrow System

What is this project?

This is a blockchain-based platform that connects **companies (sponsors)** who want their software tested, with **security researchers (hunters)** who find vulnerabilities — all without trusting a middleman.

When a sponsor posts a bounty and locks ETH, the money is held securely by a smart contract on Ethereum. The hunter only gets paid when their bug report is approved. If there's a disagreement, a neutral arbiter resolves the dispute. Everything is automated, transparent, and tamper-proof.

The Problem it Solves

Traditional bug bounty platforms like HackerOne or Bugcrowd charge high fees, can delay payments for months, and hunters have to trust the company will actually pay. With this system, the ETH is locked upfront — the hunter knows the money is there before they even start working.

The 3 Smart Contracts

BugBountyToken (BBT) — A reward token given to hunters every time they get paid. Acts like a reputation badge — the more bugs you find, the more BBT you earn.

BugBountyEscrow — The heart of the system. It locks ETH, manages the full bounty lifecycle (create → fund → submit → approve/dispute), and automatically pays the hunter when approved.

BugBountyRegistry — A public leaderboard and directory. Stores bounty listings, severity tags, and tracks each hunter's reputation score on-chain for everyone to see.

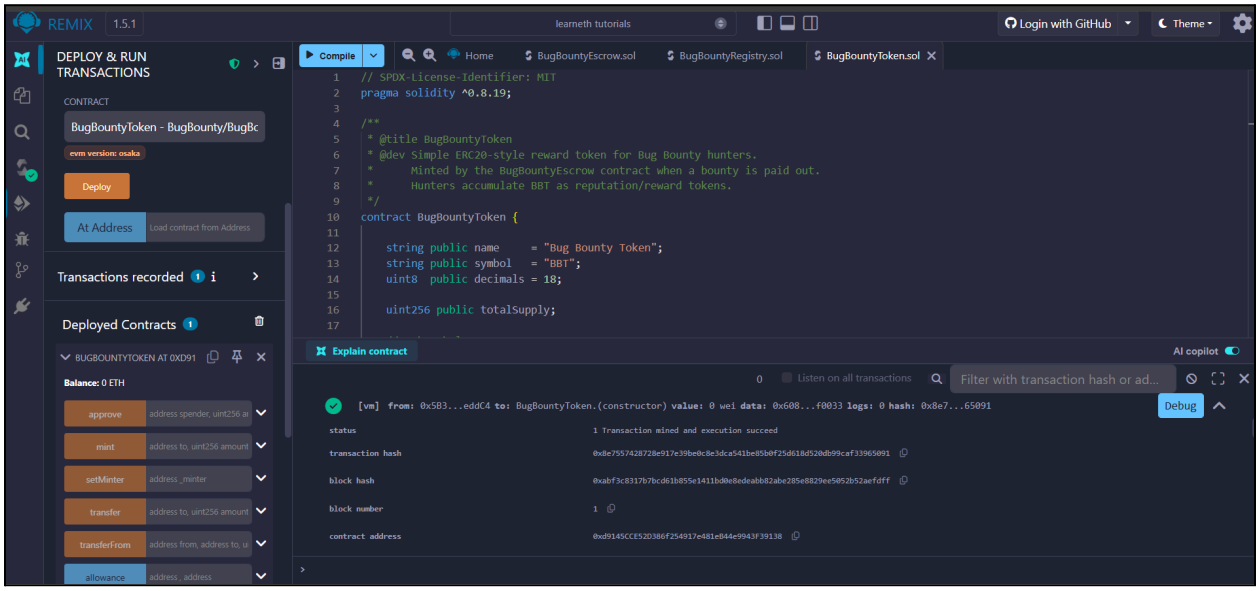
Who uses it?

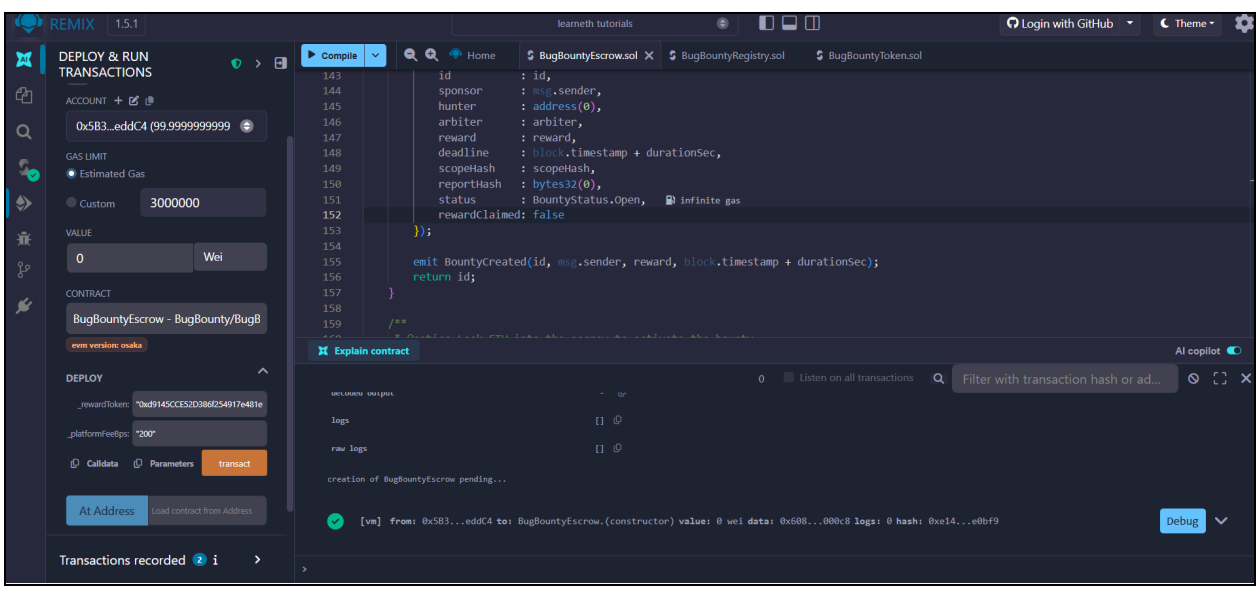
Role	What they do
🏢 Sponsor	Posts a bounty, locks ETH, reviews reports
🕵️ Hunter	Finds bugs, submits reports, earns ETH + BBT tokens
⚖️ Arbiter	Resolves disputes fairly between sponsor and hunter
👑 Platform Owner	Collects a small fee, manages the system

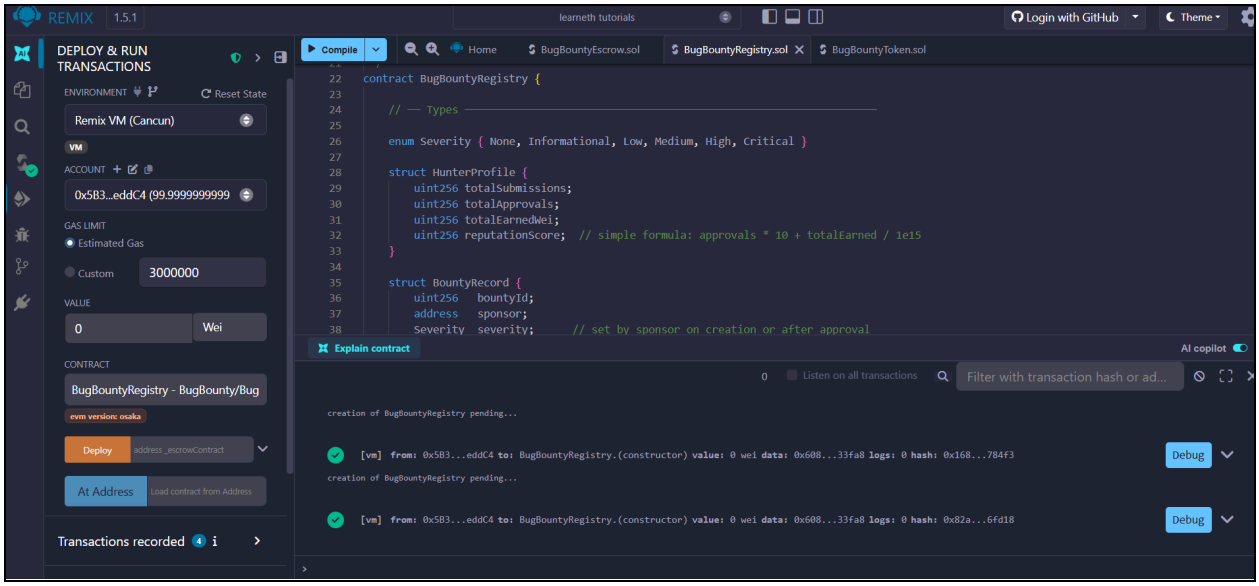
Tech Stack

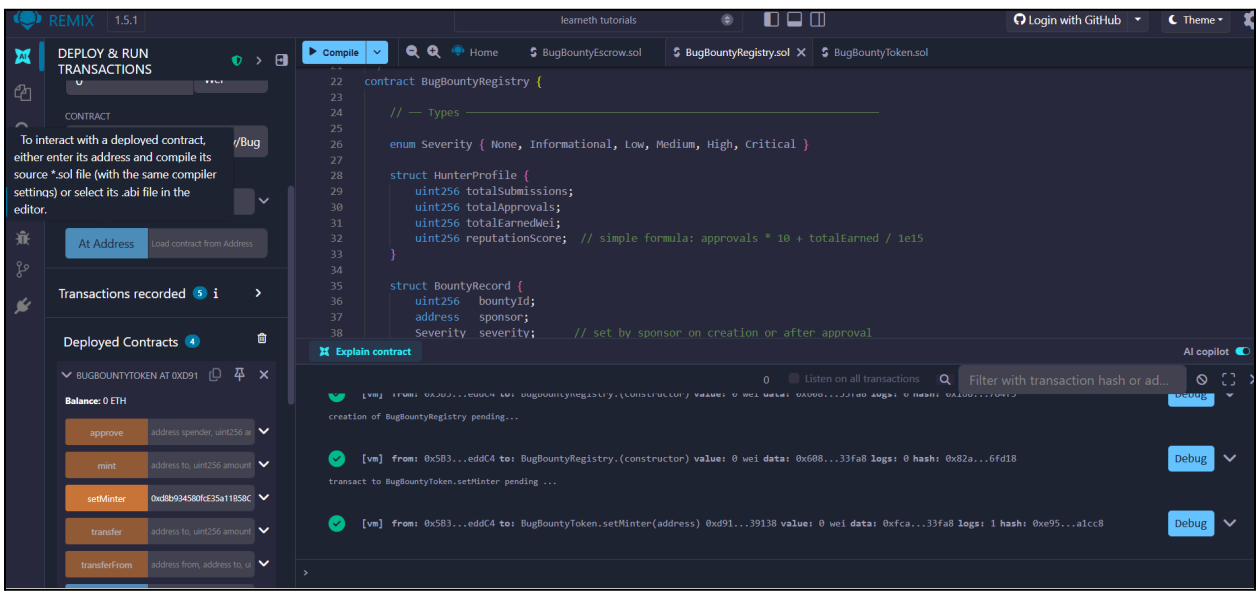
Built with **Solidity 0.8.19** on the **Ethereum** blockchain. Deployable on any EVM-compatible network (Ethereum, Polygon, Base, etc.) and fully testable on **Remix IDE** with no real money needed.

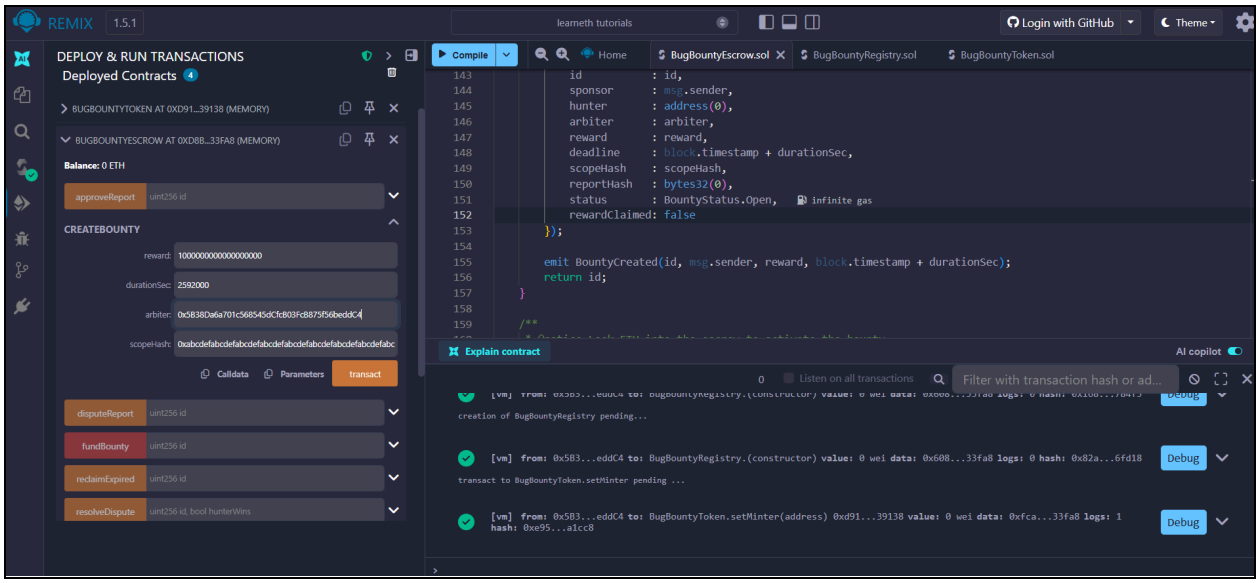
Implementation:-

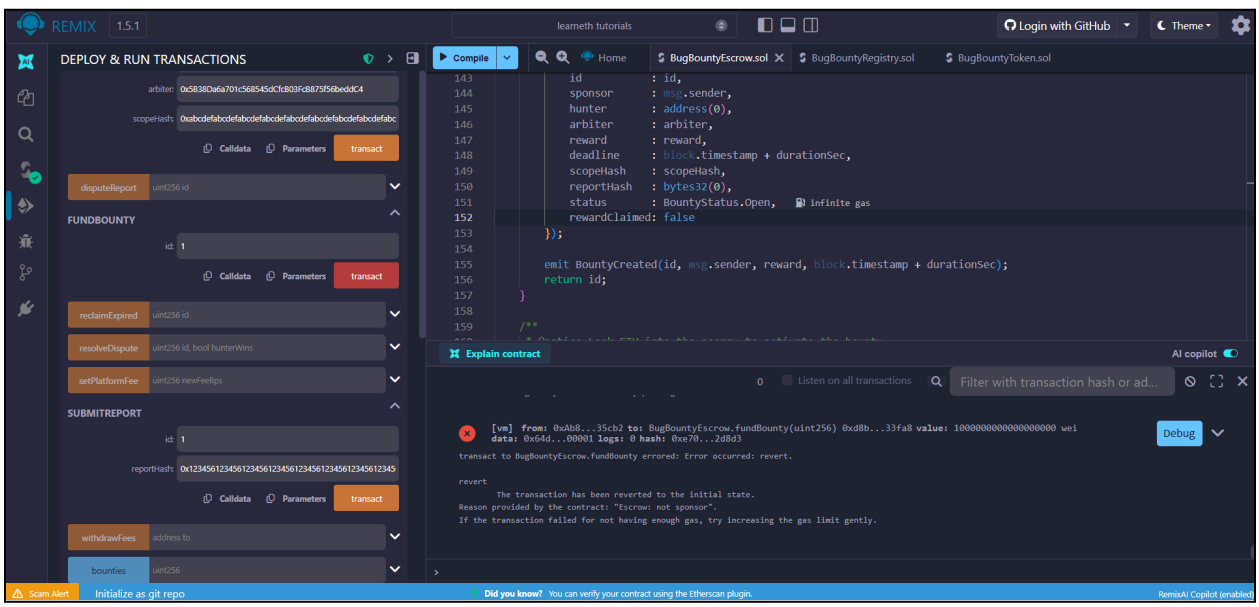




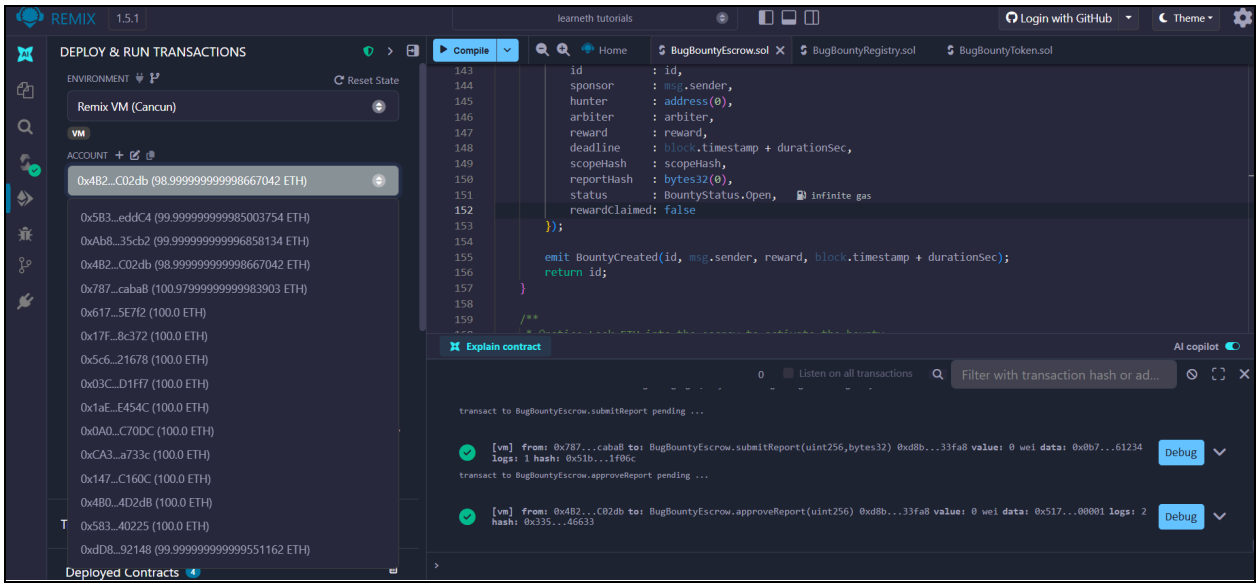

REMIX 1.5.1
learneth tutorials
Login with GitHub
Theme
DEPLOY & RUN TRANSACTIONS
ENVIRONMENT
Reset State
Remix VM (Cancun)
VM
ACCOUNT
0x5B3...eddC4 (99.9999999999
GAS LIMIT
Estimated Gas
Custom
3000000
VALUE
0
Wei
CONTRACT
BugBountyRegistry - BugBounty/Bug
evm version: osaka
Deploy
address _escrowContract
At Address
Load contract from Address
Transactions recorded 4
Compile
Home
BugBountyEscrow.sol
BugBountyRegistry.sol
BugBountyToken.sol
contract BugBountyRegistry {
// — Types
enum Severity { None, Informational, Low, Medium, High, Critical }
struct HunterProfile {
uint256 totalSubmissions;
uint256 totalApprovals;
uint256 totalEarnedWei;
uint256 reputationScore; // simple formula: approvals * 10 + totalEarned / 1e15
}
struct BountyRecord {
uint256 bountyId;
address sponsor;
Severity severity; // set by sponsor on creation or after approval
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or ad...
creation of BugBountyRegistry pending...
[vm] from: 0x5B3...eddC4 to: BugBountyRegistry.(constructor) value: 0 wei data: 0x608...33fa8 logs: 0 hash: 0x168...784f3
Debug
creation of BugBountyRegistry pending...
[vm] from: 0x5B3...eddC4 to: BugBountyRegistry.(constructor) value: 0 wei data: 0x608...33fa8 logs: 0 hash: 0x82a...6fd18
Debug


REMIX 1.5.1
learneth tutorials
Login with GitHub
Theme
DEPLOY & RUN TRANSACTIONS
CONTRACT
To interact with a deployed contract, either enter its address and compile its source *.sol file (with the same compiler settings) or select its .abi file in the editor.
At Address
Load contract from Address
Transactions recorded 5
Deployed Contracts 4
BUGBOUNTYTOKEN AT 0XD91
Balance: 0 ETH
approve
address spender, uint256 a
mint
address to, uint256 amount
setMinter
0xd8b934580fcE35a11B58C
transfer
address to, uint256 amount
transferFrom
address from, address to, u
Compile
Home
BugBountyEscrow.sol
BugBountyRegistry.sol
BugBountyToken.sol
contract BugBountyRegistry {
// — Types
enum Severity { None, Informational, Low, Medium, High, Critical }
struct HunterProfile {
uint256 totalSubmissions;
uint256 totalApprovals;
uint256 totalEarnedWei;
uint256 reputationScore; // simple formula: approvals * 10 + totalEarned / 1e15
}
struct BountyRecord {
uint256 bountyId;
address sponsor;
Severity severity; // set by sponsor on creation or after approval
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or ad...
creation of BugBountyRegistry pending...
[vm] from: 0x5B3...eddC4 to: BugBountyRegistry.(constructor) value: 0 wei data: 0x608...33fa8 logs: 0 hash: 0x82a...6fd18
Debug
transact to BugBountyToken.setMinter pending ...
[vm] from: 0x5B3...eddC4 to: BugBountyToken.setMinter(address) 0xd91...39138 value: 0 wei data: 0xfca...33fa8 logs: 1 hash: 0xe95...a1cc8
Debug





After the sponsor has approved the value the value raises



Conclusion - The Decentralized Bug Bounty Escrow System demonstrates how blockchain technology can solve real trust problems in the cybersecurity industry. By replacing middlemen with smart contracts, it ensures that hunters always get paid fairly and sponsors only release funds when satisfied — all enforced automatically by code, not promises.

With three interconnected contracts handling payments, reputation, and dispute resolution, this system creates a transparent and tamper-proof environment where security researchers can work with confidence. The addition of BBT reward tokens further incentivizes hunters by building their on-chain reputation with every successful submission.

This project proves that decentralized systems are not just a concept — they are practical, deployable tools that can reshape how industries handle trust, payments, and accountability. As Web3 continues to grow, platforms like this represent the future of fair and transparent collaboration between companies and the security community.